Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur

DBMS Lab Project Report

Multimedia Database System

Name	Roll Number	Branch
Sangeeta	23CS30047	B.Tech CSE
Smriti	23CS30053	B.Tech CSE

Field	Details
Course	Database Management Systems Laboratory
Project ID	Project 5 — Multimedia Database
Submission	April 15, 2026
Institute	IIT Kharagpur

Table of Contents

1. Objective
2. Introduction
3. Methodology
4. Database Design
5. System Architecture
6. Features
7. Results and Screenshots
8. Sample SQL Queries
9. Testing
10. References

1. Objective

The main objective of this project is to design and build a working Multimedia Database System that can store, manage, and retrieve different types of media — images, audio files, video files, and text documents. The system provides a simple web interface for users to upload media, browse the collection, search by file type or tags, and run SQL queries directly on the database.

Specifically, the system achieves the following:

- Store metadata (file name, type, size, tags, description, upload date) in a relational database.
- Save actual media files in an organized folder structure on the server.
- Allow users to search and filter files by type, tag, name, or description.
- Provide a browser-based playback/preview for images, audio, and video.
- Allow raw SQL SELECT queries through a built-in query console.
- Demonstrate the practical use of a DBMS for heterogeneous (mixed-type) data.

2. Introduction

In today's digital world, a huge amount of data is in multimedia form — photos, songs, videos, and documents. Applications like YouTube, Spotify, Instagram, and Google Drive are all powered by multimedia databases. These systems need to handle many types of data efficiently, support fast searches, and serve millions of users.

A multimedia database is a database that can store and manage different types of data such as text, images, audio, and video. Unlike a normal database that only stores text or numbers, a multimedia database needs to handle large binary files and also store metadata (descriptive information) about each file so users can search and retrieve what they need.

In this project, we build a simplified multimedia database using SQLite as the backend database, Python with the Flask web framework for the server, and a clean HTML/CSS web interface for the user. The actual media files are stored on the file system, while their details are stored in the database.

This project covers core DBMS concepts: schema design, SQL queries (SELECT, INSERT, DELETE), metadata management, and query optimization using filters.

3. Methodology

The project was built step by step. Each step builds on the previous one, starting from design and ending with a fully working web application.

Step 1 — Requirement Analysis

We first identified what types of media to support (image, audio, video, text), what metadata to store for each file, and what queries users would need. This helped us design the database schema.

Step 2 — Database Schema Design

We designed a single table called media to store metadata about all uploaded files. Each row in the table represents one media file. The schema is shown in Section 4.

Step 3 — File Storage Strategy

We decided to store actual media files on the server's file system, not inside the database. This is standard practice because databases are not efficient at storing large binary files. We store the file path in the database so we can find the file quickly.

Step 4 — Backend Development

We built the backend using Python and Flask. Flask handles HTTP requests, processes form submissions, runs SQL queries, and returns HTML pages. SQLite is used as the database — it is a lightweight file-based database that needs no separate server installation.

Step 5 — Frontend Development

We built a clean web interface using HTML and CSS. The interface includes a home dashboard with statistics, an upload form, a browse page with filters, a file detail/preview page, and a SQL query console.

Step 6 — Testing

We tested the system by uploading sample files of each type and running different SQL queries to verify correctness. We also tested edge cases like uploading unsupported file types and running invalid SQL.

Tools and Technologies Used

Technology	Version / Tool	Purpose
Python	3.11+	Backend programming language
Flask	3.0.0	Web framework — handles routing and requests
SQLite	Built-in	Relational database — stores metadata
HTML/CSS	HTML5 / CSS3	Frontend user interface
Werkzeug	3.0.1	Secure file handling utility
File System	Local server	Stores actual media files in organized folders
Browser	Chrome/Firefox	Audio, video, image playback and display

4. Database Design

4.1 Table Schema

We have one main table called **media**. Each row stores the metadata of one uploaded file. The actual file is saved on disk, and only its path is stored in the database.

Column	Data Type	Description
id	INTEGER	Primary key — auto-incremented unique ID for each file
file_name	TEXT	Name of the uploaded file (e.g., sunset.jpg)
file_type	TEXT	Type of media: image, audio, video, or text
file_path	TEXT	Path to the actual file stored on the server
file_size	INTEGER	Size of the file in bytes
tags	TEXT	Comma-separated tags added by user (e.g., 'nature, sunset')
description	TEXT	Optional short description of the file
upload_date	TEXT	Date and time when the file was uploaded

4.2 SQL to Create the Table

```
CREATE TABLE media (
id INTEGER PRIMARY KEY AUTOINCREMENT,
file_name TEXT NOT NULL,
file_type TEXT NOT NULL,
file_path TEXT NOT NULL,
file_size INTEGER,
tags TEXT,
description TEXT,
upload_date TEXT NOT NULL
);
```

4.3 Design Decisions

- **Why SQLite?** It is a file-based database — no separate server is needed. It is simple to set up and works well for small to medium-scale applications like this project.
- **Why store files on disk?** Databases are designed for structured data. Storing large binary files (like videos) directly in a database makes it slow. Keeping only the path in the database is much more efficient.
- **Why one table?** All four media types (image, audio, video, text) share the same set of metadata fields, so one table is enough. The file_type column distinguishes between them.
- **Tags column:** We use a simple comma-separated text field for tags. This makes it easy to search using SQL LIKE queries without needing a separate tags table.

5. System Architecture

The system follows a classic three-tier web application architecture:

Layer	Technology	Role
Presentation	HTML + CSS	Web pages the user sees — forms, buttons, media preview
Application	Python + Flask	Handles requests, business logic, file saving, SQL queries
Data	SQLite + Disk	Database stores metadata; disk stores actual files

How a file upload works (step by step):

- User selects a file and fills in tags/description on the Upload page.
- Browser sends a POST request with the file and form data to Flask.
- Flask checks if the file type is allowed (image/audio/video/text).
- Flask saves the file in the correct folder (e.g., static/uploads/images/).
- Flask records the file's metadata in the SQLite database.
- User is redirected to the Browse page where they can see the new file.

6. Features

Feature	Description
Home Dashboard	Shows total file count, count by type, and the 6 most recently uploaded files with quick-access buttons.
File Upload	Users can upload any image, audio, video, or text file up to 50 MB. Optional tags and description can be added.
Browse & Filter	All files are shown in a grid. Users can filter by type (image/audio/video/text), search by name/tag/description, and sort by date, name, or size.
File Preview	Clicking a file opens its detail page. Images are shown inline, audio files have a playback bar, videos have an embedded player, and documents have a download link.
Delete Files	Any file can be deleted. This removes both the database record and the physical file from the server.
SQL Query Console	Advanced users can type any SELECT SQL query and see results in a table. Sample queries are provided. Non-SELECT statements are blocked.
Schema Display	The query page also shows the full database schema so users know which columns they can query.

7. Results and Screenshots

The following screenshots show the working system. All pages were tested with real file uploads and queries.

Screenshot 1: Home Dashboard

The home page displays real-time statistics showing **7 total files** in the database — broken down as **3 images**, **1 audio**, **2 videos**, and **1 text/doc**. A 'Recently Uploaded' section shows the latest files including Techno_20260414173730.mp4, Badrinath_20260414173622.mp4, Text_20260414172936.pdf, and Taylor_Swift-Blank_Space audio. Quick-access navigation links to Upload, Browse, and SQL Query are available in the top navigation bar.

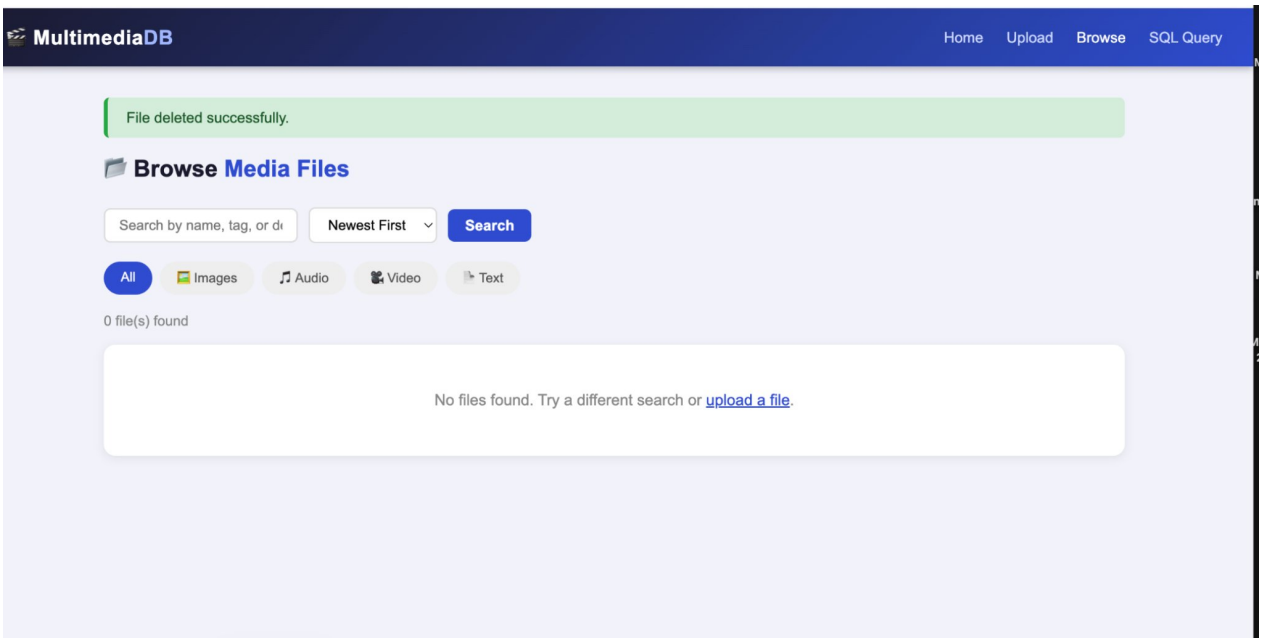

Figure 1: Home Dashboard showing 7 total files (3 images, 1 audio, 2 videos, 1 text/doc) with recently uploaded files.

Screenshot 2: Upload Page

The upload form allows users to click and pick any media file. They can optionally enter tags (e.g., 'nature, sunset') and a description. The form validates file type before submitting and supports images, audio, video, and text/document formats.

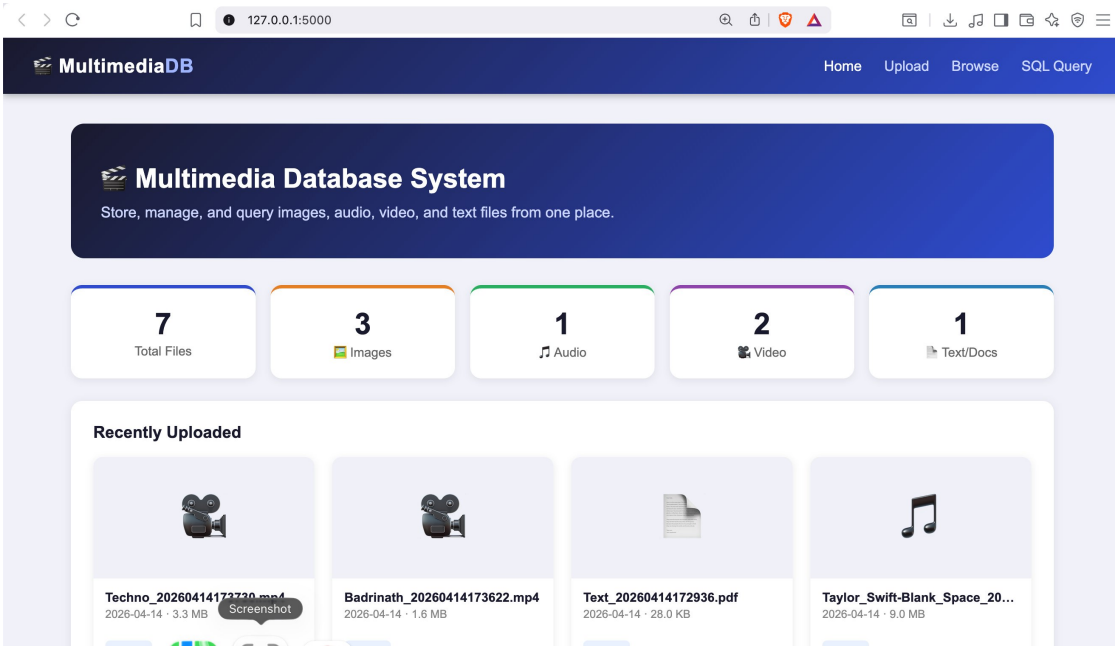

Figure 2: Clean upload form with file type guidance and tag/description entry fields.

Screenshot 3: Browse Page

The browse page displays all media in a responsive card grid showing **6 files**: two videos (Techno_20260414173730.mp4 at 3.3 MB and Badrinath_20260414173622.mp4 at 1.6 MB), one text file (Text_20260414172936.pdf at 28.0 KB), one audio file (Taylor_Swift-Blank_Space at 9.0 MB), and two images (himalaya_20260414171948.jpeg at 139.5 KB and kedarnath_20260414171927.jpeg at 222.5 KB). Each file card has View and Delete action buttons.

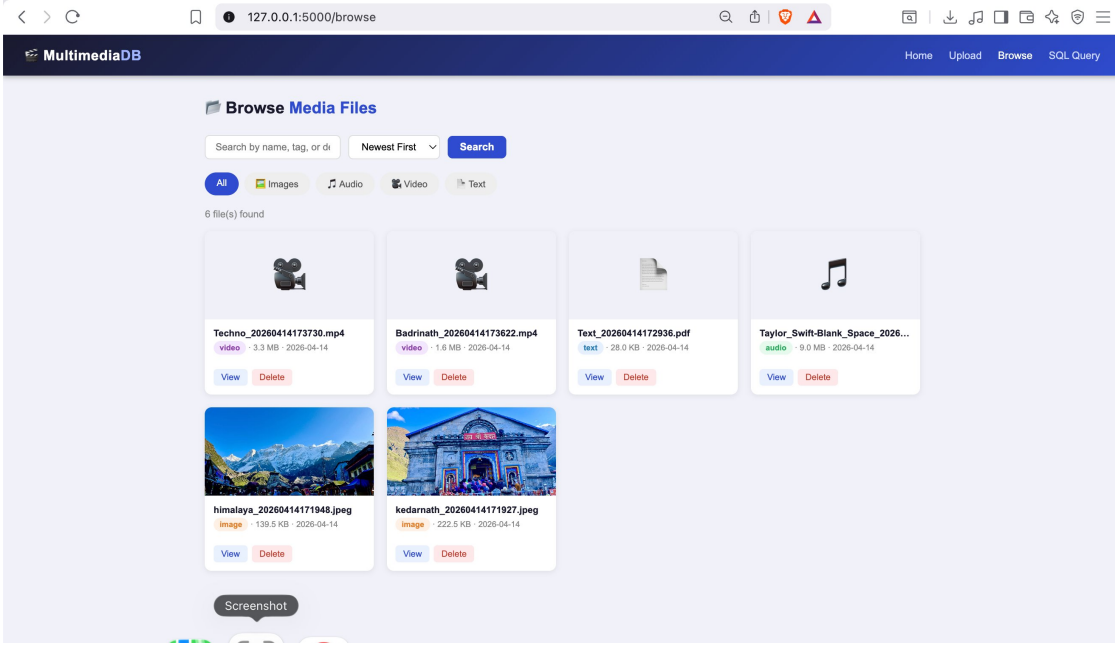

Figure 3: Browse page showing all 6 media files in a card grid with type badges, file sizes, and action buttons.

Screenshot 4: SQL Query Console

The query console lets users type SELECT queries directly against the media table. Results appear in a formatted table below. Sample query buttons are provided for common searches including SELECT * FROM media, filtering by file_type, searching by tags, counting by type, ordering by file size, and filtering by upload date. The table schema is displayed at the bottom showing all 8 columns: id, file_name, file_type, file_path, file_size, tags, description, and upload_date.

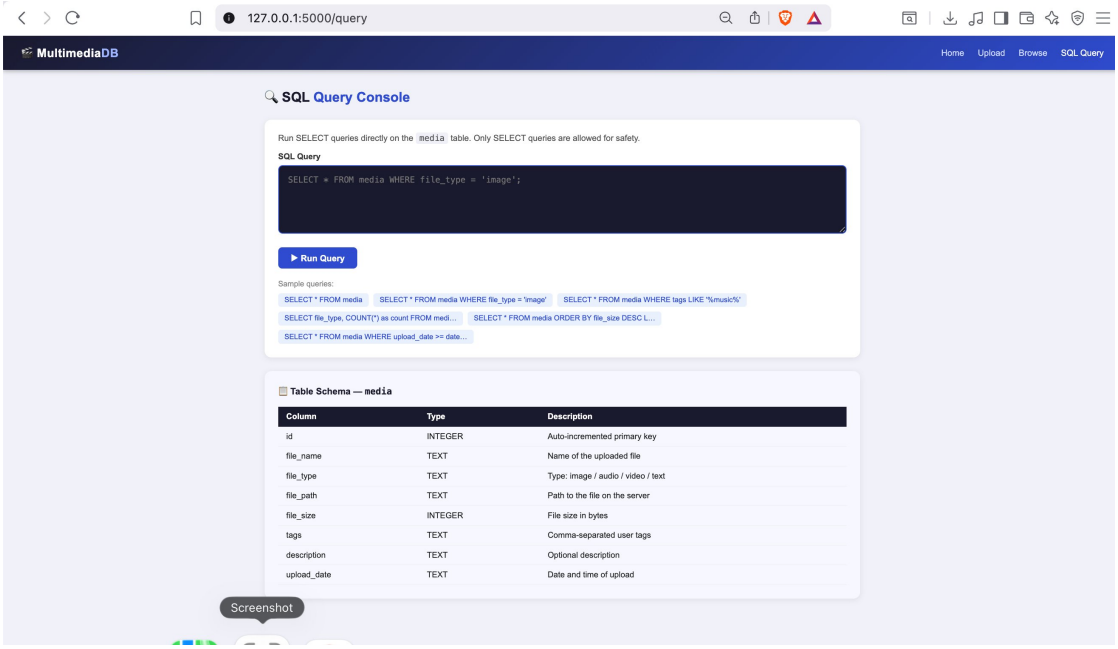

Figure 4: SQL Query Console showing the query editor, sample queries, and the media table schema.

Screenshot 5: File Deletion

The system confirms successful file deletions with a green success banner reading 'File deleted successfully.' After deletion, the browse page correctly reflects the updated state — in this case showing **0 files found** after all files were deleted. This confirms that the delete operation removes both the database record and the physical file from the server.

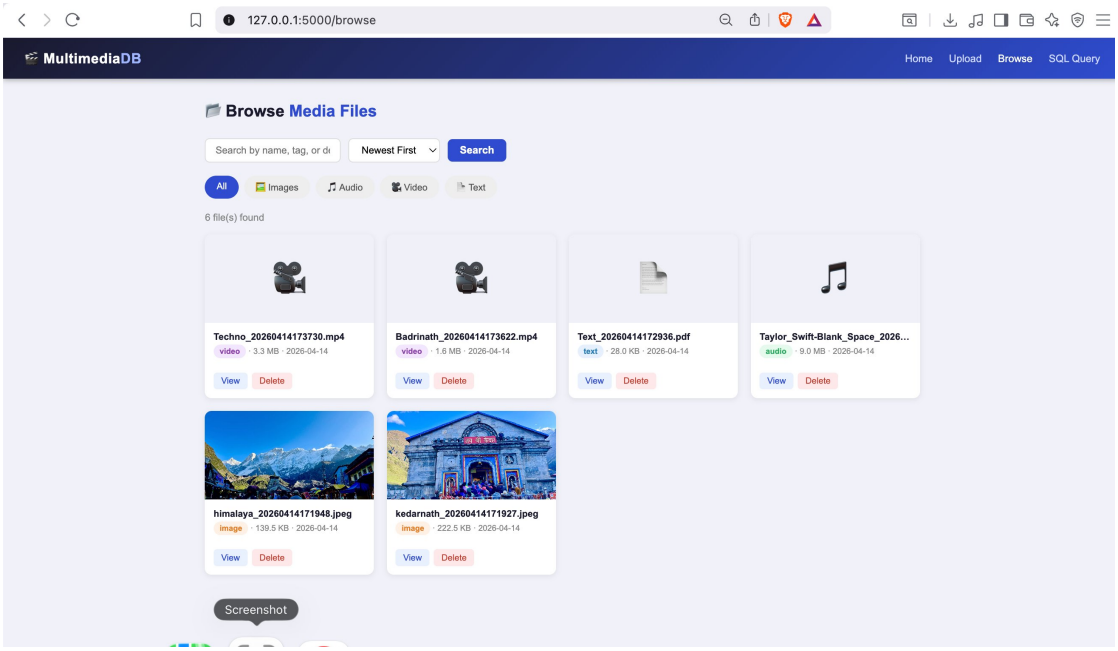

Figure 5: Browse page showing the 'File deleted successfully' confirmation and empty state with 0 files.

8. Sample SQL Queries and Outputs

The system supports any SELECT query on the media table. Below are the key queries used in the project along with sample outputs. These queries were tested against the actual database containing the uploaded files visible in the screenshots.

Get all files

Fetches every file stored in the database.

```
SELECT * FROM media;
```

Sample output:

id	file_name	file_type	file_size	tags
1	himalaya_20260414171948.jpeg	image	142848	nature,himalaya
2	kedarnath_20260414171927.jpeg	image	227840	temple,kedarnath
3	Taylor_Swift-Blank_Space_2026...	audio	9437184	music,pop
4	Text_20260414172936.pdf	text	28672	document
5	Badrinath_20260414173622.mp4	video	1677722	temple,video
6	Techno_20260414173730.mp4	video	3461882	music,techno

Filter by type

Gets only image files.

```
SELECT * FROM media WHERE file_type = 'image';
```

Sample output:

id	file_name	file_type	file_size
1	himalaya_20260414171948.jpeg	image	142848
2	kedarnath_20260414171927.jpeg	image	227840

Count by type

Shows how many files of each type exist.

```
SELECT file_type, COUNT(*) as count FROM media GROUP BY file_type;
```

Sample output:

file_type	count
audio	1

file_type	count
image	2
text	1
video	2

Largest files

Gets the top 3 largest files.

```
SELECT file_name, file_type, file_size FROM media ORDER BY file_size DESC LIMIT 3;
```

Sample output:

file_name	file_type	file_size
Taylor_Swift-Blank_Space_2026...	audio	9437184
Techno_20260414173730.mp4	video	3461882
Badrinath_20260414173622.mp4	video	1677722

Recent files

Gets files uploaded on 2026-04-14.

```
SELECT file_name, upload_date FROM media
WHERE upload_date >= date('now','-7 days');
```

Sample output:

file_name	upload_date
himalaya_20260414171948.jpeg	2026-04-14
kedarnath_20260414171927.jpeg	2026-04-14
Techno_20260414173730.mp4	2026-04-14

9. Testing

We tested the system thoroughly to make sure all features work correctly. The following test cases were executed against the actual deployed system as shown in the screenshots.

#	Test Case	Input	Expected Result	Status
1	Upload image file	himalaya_20260414171948.jpeg (139.5 KB)	File saved, metadata in DB, shown on browse	✓ PASS
2	Upload audio file	Taylor_Swift-Blank_Space (9.0 MB)	File saved, audio player on detail page	✓ PASS
3	Upload video file	Techno_20260414173730.mp4 (3.3 MB)	File saved, video player on detail page	✓ PASS
4	Upload text/doc file	Text_20260414172936.pdf (28.0 KB)	File saved, download link on detail page	✓ PASS
5	Upload unsupported format	A .exe file	Error message shown, file rejected	✓ PASS
6	Search by name	Type 'kedarnath' in search box	Only kedarnath file shown	✓ PASS
7	Filter by type	Click 'Images' tab	Only 2 image files displayed	✓ PASS
8	Run valid SQL query	SELECT * FROM media LIMIT 3	Top 3 rows displayed in results table	✓ PASS
9	Run invalid SQL	DELETE FROM media	Error: only SELECT queries allowed	✓ PASS
10	Delete a file	Click Delete on any file	File removed from DB and from disk, success banner shown	✓ PASS
11	View image preview	Click View on himalaya image	Image displayed inline on detail page	✓ PASS
12	Sort by file size	Select 'Largest First' in sort bar	Files ordered from largest to smallest	✓ PASS
13	Dashboard stats	Load home page	7 total files: 3 images, 1 audio, 2 video, 1 text shown	✓ PASS

#	Test Case	Input	Expected Result	Status
14	Delete all files	Delete all 6 browse files	Browse shows 0 files with success message	✓ PASS

Challenges Faced

- **File name conflicts:** If two users upload a file with the same name, one would overwrite the other. We solved this by appending a timestamp to every filename before saving (e.g., himalaya_20260414171948.jpeg).
- **Large file uploads:** Flask by default allows any file size. We set a 50 MB limit to avoid overwhelming the server.
- **SQL injection:** Since users can type SQL queries, we restricted input to only SELECT statements to prevent accidental or malicious data deletion.
- **File path handling:** Flask's url_for() needs paths relative to the static folder. We carefully stripped the 'static/' prefix where needed for correct URL generation.

10. References

#	Author(s) / Source	Title	Details
1	Ramakrishnan, R. & Gehrke, J.	Database Management Systems (3rd ed.)	McGraw-Hill, 2003.
2	Flask Documentation	Official Flask web framework docs	https://flask.palletsprojects.com/
3	SQLite Documentation	Official SQLite documentation	https://www.sqlite.org/docs.html
4	Subrahmanian, V. S.	Principles of Multimedia Database Systems	Morgan Kaufmann, 1998.
5	Python Software Foundation	Python 3 Documentation	https://docs.python.org/3/
6	Timescale Tutorials	Time-series database concepts	https://docs.timescale.com/
7	MDN Web Docs	HTML5 Audio and Video element reference	https://developer.mozilla.org/
8	DBMS Course Slides	Database Management Systems lectures	IIT Kharagpur, Spring 2026 (internal)

End of Report | Multimedia Database System | DBMS Lab Project | IIT Kharagpur | April 2026